

Session 208
Enhanced Graphics

Bob Ebert
Palm OS Engineering

3Com
More connected.

Overview

- Reflection
- Disillusionment
- Improvement
  - Hardware
  - Blitter
  - Bitmaps
  - Windows
  - User Interface
  - Applications
- Questioning

In the Distant Past…

PalmSource 99

- Black and White
- …okay, really gray and green
- Low contrast displays (but cheap!)

- Grayscale: by hacking the registers of the Dragonball LCD controller
- Didn’t look great, but made cool games

Spring ’98: OS 3.0

PalmSource 99

- Some OS level support for grayscale
- Only supported API to change depth
  - So you could stop banging registers
- “Foreground” and “Background” RGB colors would draw in black, dk gray, lt gray, or white
- Gray on green in 4 levels if you had your contrast set right

Fall ’98: OS 3.1, 3.2

PalmSource 99

- Better displays! (dark gray on gold)
- EZ processor! 4-bit per pixel allows 16 shades
- For 4-bpp, back to hacking registers
  - OS 3.3 adds 4-bpp support
- Monochrome pictures look pretty good
- “Seeing Gray” white paper tells all

Today

- OS support for 1, 2, 4, and 8 bit per pixel
- Colors!
  - Real red, green, blue
  - Cyan, magenta, yellow
  - Mauve, taupe, ecru
  - Strawberry, platinum, graphite, ice…
- In the future
  - Hardware—don’t ask me

Today's Toolbox

PalmSource 99

- KISS approach
- All the old APIs still work
- New routines where they make sense
  - Keeping performance in mind
  - It is a 16Mhz 68000 after all
- Same drawing model on all devices
  - So "color" apps will work on Palm III™ organizer
- Minimal use of color in user interface
  - The data, it's all about the data

Disillusionment

- No Polygons, Arcs, or Regions
- No Color Patterns
- No Multi-Color Fonts
- No Alpha Channels, Direct Mode
- No Encapsulated Drawings (postscript)
- No Flood Fill
- No Sub-Pixel Rendering of Fonts
- No Panning
- No Sprites, Ray Tracing, Texture Mapping

Improvement

- **Hardware**
- Blitter
- Bitmaps
- Windows/Drawing
- User Interface
- Applications

Hardware

- Abstraction Layer in place
  - Common set of low level routines
  - Same System code runs on different hardware
- Not for use by applications
- Higher level OS routines cover everything you need (we hope)
  - Ensure that higher layers stay in sync
- Never write to registers! (Please)
  - Use HAL if you have to

Hardware Abstraction Layer

- Manages:
  - Screen base address
  - Screen depth (get/set)
  - VRAM size/location (if any)
  - Contrast, Brightness
  - Palette (if possible)
  - Power, wake, sleep
- HwrDisplay.h
  - With Source license

Improvement

- Hardware
- **Blitter**
- Bitmaps
- Windows/Drawing
- User Interface
- Applications

Blitter

- Used to be called ScrDriver
  - Was never “screen” specific
- Renders into any bitmap
  - On screen or off
- Supports 1, 2, 4, and 8-bpp bitmaps
  - Source and destination, all combinations

- Handles Depth Conversion
- Handles Color Translation
- Handles More Transfer Modes

Blitter: Basic Operations

PalmSource 99

- Bitmaps: BltCopyRectangle
- Characters: BltDrawChars
- Lines: BltLineRoutine
- Rects: BltRectangleRoutine

- That's it (!)
  - Also RGB <--> index conversion
  - Also old Scr* routines

Blitter: Parameters

- Three “pen” colors:
  - Foreground
  - Background
  - Text
- Patterns:
  - None, black, white
  - Custom: 1 bit deep
  - Foreground and background colors apply
- Underlines for text
  - Chars are text color
  - Underline is foreground color

Blitter: General API

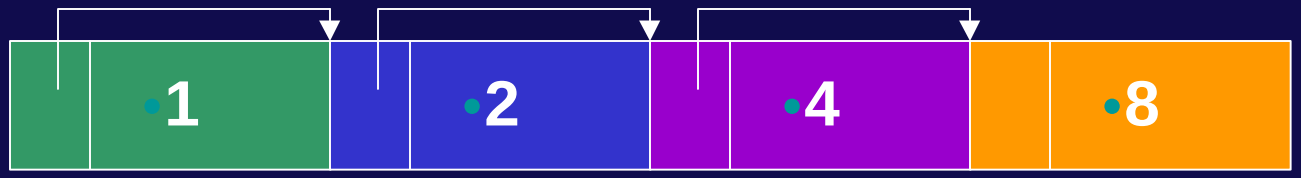
- All Functions Take:
  - Destination bitmap
  - Drawing state (pen, pattern, font, etc.)
  - Clipping
- Function specific:
  - String
  - Line start and end points
  - Rectangle sides and rounding
  - Source bitmap

Improvement

- Hardware
- Blitter
- **Bitmaps**
- Windows/Drawing
- User Interface
- Applications

Bitmaps

- Families
  - Include zero or one image at each depth
- Each image (depth) contains:
  - Width, Height, Depth
  - Compression
  - Transparency
  - Color Table
  - Bits
  - Flags, versioning, rowbytes



Bitmaps: API

- New interface: Bitmap.h
- Bitmap structure is public, supported
  - Format for ‘Tbmp’ and ‘tAIB’ Resources
- Functions to Create, Delete
- Function to Compress
  - Scan line, RLE, maybe more later
- Accessor functions for convenience
  - Size, bits, colortable

Bitmaps: Color Table

- Not ignored!
- Structure:
  - Length (Int16)
  - Array of:
    - Index (Byte)
    - Red (Byte)
    - Green (Byte)
    - Blue (Byte)
- Blitter builds translation table based on RGB
- Use table when drawing—SLOW

Bitmaps: Color Tables

PalmSource 99

- RGB to Index conversion is “best fit”
  - No automatic “palette management”
- Grayscale: best fit is by luminance
- Color: best fit in RGB space (good enough)

- For best results, reprogram color table for display
- Then render sources that are based on the same colortable

Bitmaps: Transparency

- Our solution for masking
- Easier to edit source data
- Takes about the same time to render
- Uses about as much storage space

- One index value per bitmap can be ‘invisible’
- Not very useful for 1-bpp bitmaps
- Impacts performance

Bitmaps: Depth Translation

- If more than one source (in the bitmap family)
- Use the one with the largest depth that is still less than or equal to the destination depth, or lowest depth if none
  - (It's easier to code than it is to write)
- Depth translation also uses 'best fit' color translation
- If no colortable in source image, assume system default for that image

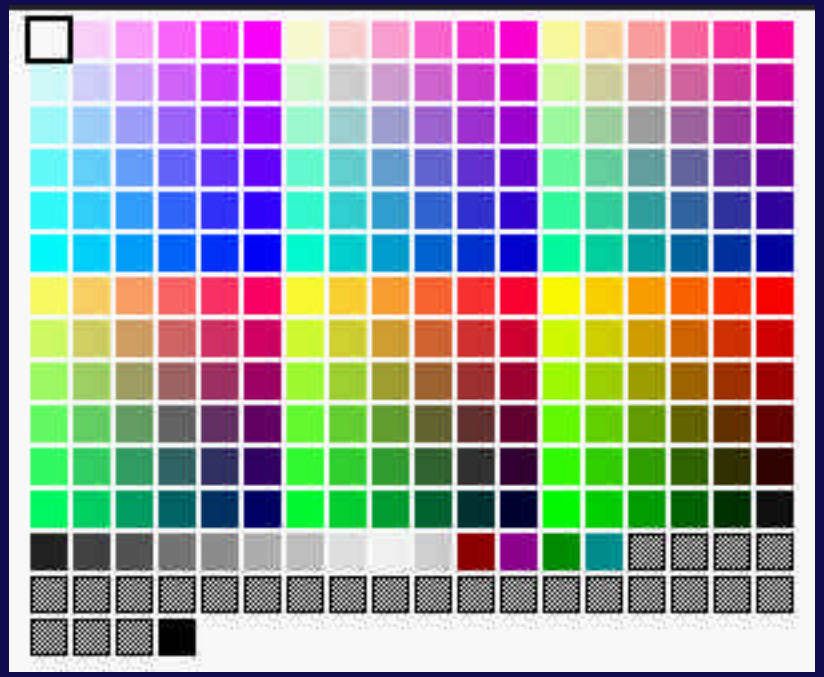
Default Color Tables (Palettes)

- 1 bpp—white and black
- 2 bpp—4 levels
- 4 bpp—16 levels
- 8 bpp:
  - White=0
  - Black = 255
  - 216 “web safe” colors
  - (00,33,66,99,CC,FF)
  - 10 more grays
  - Silver, maroon, purple, green, navy
  - 25 unassigned









Bitmaps: Compression

- No compression—version 0
- Scan Line compression—version 1
- RLE compression—version 2
- May be more in the future
  - GIF?
  - JPEG?

Bitmaps: Tools

- PalmRez (Mac) converts PICTs
- New ‘tbmf’ bitmap family resource
- Similar ‘taif’ for App Icons
  - Only ‘Tbmp’ and ‘tAIB’ on device!
- Constructor will have nice UI
- Specify image, transparency, compression, colortable separately for each depth

Improvements

PalmSource 99

- Hardware
- Blitter
- Bitmaps
- **Windows/Drawing**
- User Interface
- Applications

Window Layer: Old Stuff

- All the old calls, plus some new ones
- WindowType structure changed
  - Can’t get the screen address out of it
- Basic window management unchanged
- WinCreateOffscreenWindow still best way to get an off-screen window (same depth)
- Still uses the global DrawWindow concept

Window Layer: Old Stuff

PalmSource 99

- Things we'd like you to start doing:

- WinGetWindowExtent—use it!
  - Do not use hardcoded screen size
  - WinGetDisplayExtent if you have to
- WinDrawBitmap—use it!
  - Instead of WinCopyRectangle
- Avoid WinInvert*
  - Meaning for color is ambiguous
  - Change colors and draw again

Window Layer: New Stuff

- WinPaint*
  - Draw routines that respect transfer mode
- Push/Pop draw state
  - Saves transfer mode, colors, etc.
  - So you can change them all, then easily restore
- API to change draw state
  - Set font, pen, transfer mode, pattern
- WinGetBitmap, WinCreateBitmapWindow
  - Windows are backed by bitmaps
  - These routines let you manage that

Window Layer: New Stuff

PalmSource 99

- WinGetPixel
  - Get color index from draw window
- WinPaintPixel (Draw, Erase, Invert, etc.)
  - Write individual pixels
- WinPaintPixels, WinPaintLines
  - Draw many at once, avoid trap overhead

Window Layer: Palette Management

- WinIndexToRGB
  - Get ‘real’ RGB value for an index
- WinRGBToIndex
  - Get index for a given RGB, best fit
- WinPalette
  - Get/set color table for draw window
  - Complex API, read it twice, then read it again
- sysNotifyDisplayChangeEvent
  - When palette for screen changes

Window Layer: Screen Support

PalmSource 99

- WinScreenMode
  - Change depth
  - Maybe width and height someday?
  - Also does a notify
- WinScreenLock
  - Freeze display, drawing suspended
  - Init mode: Copy, Erase, Nothing
- WinScreenUnlock
  - Bring screen up to date

Improvements

- Hardware
- Blitter
- Bitmaps
- Windows/Drawing
- **User Interface**
- Applications

User Interface

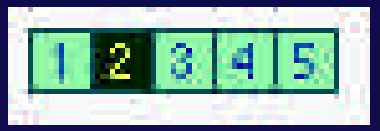
- Graphic Buttons
  - Have one or two bitmaps, animate properly
  - “Old” graphic buttons are supported, look odd
- Sliders (Horizontal only)



  - Can provide background, thumb graphics
- Other controls draw in a set of colors
- UI Color Table
  - A set of colors used by the UI
  - Tried to create a consistent visual language
  - While keeping the list short
  - Some customization allowed

User Interface Colors

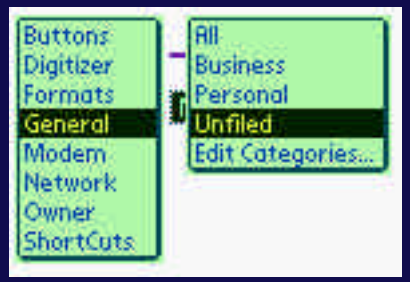
- UI Objects (Controls)
  - Frame
  - Fill
  - Foreground
  - Selected Fill
  - Selected Foreground



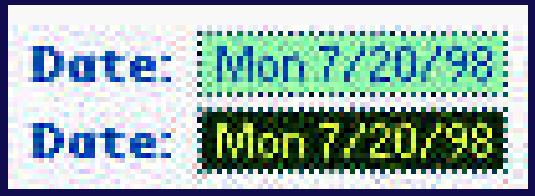












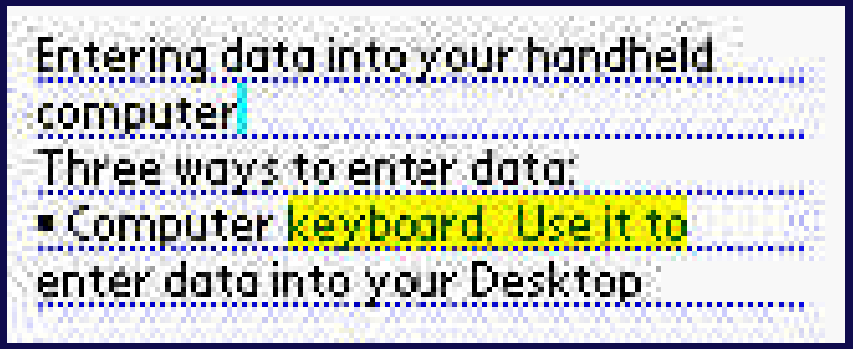
User Interface Colors

- Menus
  - Frame
  - Fill
  - Foreground
  - Selected fill
  - Selected foreground



User Interface Colors

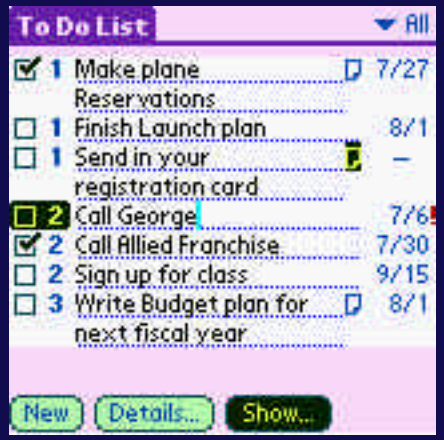
- Fields
  - Background
  - Text
  - TextLines
  - Caret
  - TextHighlightBackground
  - TextHighlightForeground

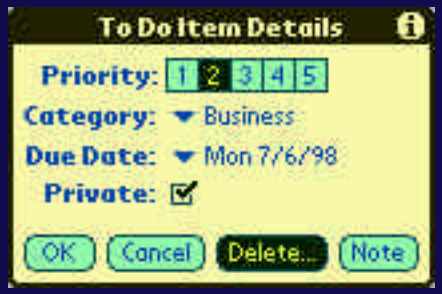


  - FEP{Raw|Converted}{Text|Background}
  - FEPUnderline
  - (FEP still uses underline if FEP colors match normal colors)

User Interface Colors

- Forms
  - Frame
  - Fill
- Dialogs
  - Frame
  - Fill
- Alerts
  - Frame
  - Fill







User Interface Colors

PalmSource 99

- Others
  - OK, Caution, Warning



User Interface Colors

- Can’t draw one control different from others
- …no space to store data without breaking compatibility
- You can try changing UI color, drawing, changing it back
  - But it won’t work reliably
- So, don’t—Leave colors the same

UI Customization

- One UI color set for each depth
  - So black and white color scheme can differ from grayscale and from color
- Loaded from user preferences DB
- Or from OS resource if no pref
- Color scheme (personality) editors?

Improvements

- Hardware
- Blitter
- Bitmaps
- Windows/Drawing
- User Interface
- **Applications**

Applications

- Very few app specific changes
- Launcher handles color icons
- Apps use graphic buttons
  - Calculator, Date Book, HotSync
- A few color icons here and there
- We’d like feedback, and your apps in color!

(Your) Applications

PalmSource 99

- Can develop them using existing tools
  - Ask the tools guys about color support!
- Test is Palm OS® Emulator (Poser)
  - Use the debug ROMs!
- Test in Mac Simulator
  - SimMain.cp has boot depth global
- Compatibility
  - ROM version test
  - SysGetTrapAddress test
  - WinScreenMode test

Questioning

- Any Questions?


PalmSource™ 99
3Com
More connected.